
现在别让我停下： 基于嵌入的 LLM 调度

Rana Shahout¹

Eran Malach¹

Chunwei Liu²

Weifan Jiang¹

Minlan Yu¹

Michael Mitzenmacher¹

¹ 哈佛大学

² 麻省理工学院

摘要

高效调度对交互式大语言模型 (LLM) 应用至关重要，因为较低的请求完成时间直接影响用户参与度。最短剩余处理时间 (SRPT) 等按作业大小调度的算法，利用已知或估计的请求大小，允许新到达、服务时间更短的作业抢占当前作业，以降低平均请求完成时间。然而，将这类调度用于 LLM 系统有两项主要挑战。首先，根据提示准确预测输出长度十分困难，而且往往消耗较多资源，在许多系统中并不实用。因此，先进的 LLM 系统通常默认采用先来先服务，可能造成队头阻塞并降低效率。其次，抢占增加额外内存开销，因为系统必须保留尚未完成、已经被抢占的请求的中间状态。

本文提出 TRAIL，直接从目标 LLM 本身获取输出长度预测。每生成一个输出 token，我们就复用模型内部结构的嵌入，输入轻量分类器，预测每个运行请求的剩余长度。基于这些预测，我们提出限制抢占的预测型 SRPT 变体，专门考虑 LLM 系统的内存开销。该策略允许请求执行早期在内存消耗较小时被抢占，而在接近完成时限制抢占，以优化资源利用。理论方面，我们在 M/G/1 队列模型下推导该 SRPT 变体的闭式公式，展示其潜在价值。系统实现结合这一抢占策略与基于嵌入的预测。相比根据序列提示进行的 BERT 预测，基于层嵌入的细化预测将平均绝对误差降至其 1/2.66。在 Alpaca 数据集上，相比先进服务系统，TRAIL 的平均延迟降至基线的 1/1.66 至 1/2.01，平均首 token 时间降至基线的 1/1.76 至 1/24.07。

1 引言

大语言模型 (LLM) 的近期发展催生了新一代交互式 AI 应用。OpenAI 的 ChatGPT (OpenAI, 2022) 就是典型例子，支持用户通过对话完成多种任务。这类应用的交互性质要求较低的请求完成时间，才能保持用户参与。用户期望近乎即时的响应，因此，高效推理服务对基于 LLM 的交互式 AI 应用至关重要。

LLM 推理请求具有独特的自回归模式，包含多次迭代；每次将新输出 token 加入输入，再生成下一个 token。Orca (Yu et al., 2022) 和 vLLM (Kwon et al., 2023) 是先进的 LLM 推理系统，采用迭代级调度，允许在每次迭代结束时加入新请求或移除已完成请求，相比请求级调度更灵活地管理执行批次。然而，它们仍按先来先服务 (FCFS) 处理请求，在高负载下导致队头阻塞 (Kaffes et al., 2019)。这一问题对 LLM 推理尤其严重，因为许多短请求可能等待其他长请求执行完毕。

按大小调度利用已知或估计的请求大小，降低请求完成时间。LLM 请求的执行时间取决于输入和输出长度，其中输出长度未知且可能变化。抢占支持动态调度：运行中的请求生成一

一个输出 token 后，可以决定继续执行，或由另一个新到达请求替换它。最短剩余处理时间 (SRPT) 就是抢占式调度策略。然而，这里的抢占会增加内存开销，以保存已启动但尚未完成请求的中间状态。LLM 在每个 Transformer 层维护键值 (KV) 缓存，存储这些状态。抢占式策略必须保留等待队列中所有已启动但未完成请求的缓存；由于模型庞大、GPU 内存有限，这可能造成内存溢出。非抢占策略，例如 FCFS，不会面临这种问题，因为其等待队列中没有已启动但未完成的请求。

近期工作提出了请求大小预测方法。 S^3 (Jin et al., 2023) 微调 BERT 模型 (Sanh, 2019)，根据输入提示预测输出序列长度。该模型资源消耗较低，但面对执行时间差异较大的请求时，准确率下降。Zheng 等 (Zheng et al., 2024) 使用另一个更小的 LLM，在调度前预测输出长度，精度高于 BERT。不过，这种方法消耗更多资源，其实际部署效率与成本效益仍需考量。同一工作还提出预先感知，要求模型预测即将生成回答的长度；但预测与生成结果可能相互影响，进而影响输出质量，因此存在限制。

本文提出 TRAIL (*Token Response and Embedding Adaptive Inference Layer* (基于 token 响应与嵌入的自适应推理层))，通过两项关键贡献改善 LLM 推理响应时间：(1) 以低开销、高精度在迭代级预测请求长度，支持按大小调度；(2) 以 token 为粒度执行考虑内存开销的抢占调度。预测方面，利用 LLM 输出的自回归性质，在每个 token 粒度上预测输出大小和请求长度。首先根据提示生成初始预测，再随序列推进逐步细化；关键在于复用 LLM 层嵌入。具体地，将中间 Transformer 层的嵌入“回收”后，输入轻量线性分类器，预测剩余序列长度。这种将模型内部层输出作为另一个机器学习预测器输入的分析方法，通常称为探针分析 (*probing*) (Belinkov, 2022; Hewitt & Liang, 2019; Hewitt & Manning, 2019)。它兼顾基于 LLM 预测的优势与计算效率，无须额外部署专门用于长度预测的 LLM。为解决抢占调度的内存约束，TRAIL 限制每个请求可被抢占的次数，提出有限抢占 SRPT。请求执行早期 KV 缓存占用较小，抢占的内存代价较低；随着执行推进，KV 缓存增大，抢占变得更昂贵，因此后期关闭抢占，避免高内存开销。

本文贡献包括：(1) 复用 LLM 嵌入的输出长度预测框架，在每次迭代动态细化预测；实验中，基于层嵌入的细化预测将平均绝对误差降至序列提示 BERT 预测的 1/2.66；(2) 使用这些预测、限制抢占的最短剩余处理时间变体；(3) 在 M/G/1 队列模型下推导该变体的闭式公式。实验还表明，预测和调度方法能够缓解队头阻塞。将 TRAIL 集成到先进 LLM 系统后，在 Alpaca 数据集上，相比该服务系统，平均延迟降至基线的 1/1.66 至 1/2.01，平均首 token 时间降至基线的 1/1.76 至 1/24.07。TRAIL 有望显著提高 LLM 推理效率与可靠性，推动更具适应性的服务系统发展。

2 背景与研究动机

基于 Transformer 的生成模型与键值缓存。 每一步中，Transformer 根据目前已生成的 token 序列，预测下一个最可能的 token。生成长度为 n 的序列需要执行 n 次迭代，每个 token 经过多个由自注意力和前馈网络组成的 Transformer 层。

在第 i 步的一次典型迭代中，模型通过自注意力计算所有先前 token $(t_0, t_1, \dots, t_{i-1})$ ，可表示为：

$$h_{\text{out}} = \text{softmax} \left(\frac{q_i \cdot K^\top}{\sqrt{d_h}} \right) \cdot V$$

其中， q_i 是表示当前 token t_i 隐藏状态的查询向量， $K, V \in \mathbb{R}^{i \times d_h}$ 分别是根据所有先前 token 得到的键矩阵与值矩阵。

为提高效率，LLM 在整个生成过程中缓存键和值矩阵 (KV 缓存)，避免每次迭代重新计算。这显著减少计算时间，但需要大量内存，其占用与层数和隐藏维度成比例。随着生成更多 token，缓存保留全部已有 token 的信息，内存需求随序列长度线性增加；因此，尤其是长序列，KV 缓存占用十分可观。例如，在 GPT-3 175B 模型中，单个长度为 512 token 的请求，至少需要 2.3 GB 内存存储键值对。GPU 内存容量限制 KV 缓存大小，也使高效实现抢占式调度变得困难。

迭代级调度。 迭代级调度不同于传统请求级调度。请求级调度将整批请求运行至完成：提前完成的请求仍需等待整批结束，新请求也必须在队列中等待下一批开始。迭代级调度 (Yu et al., 2022) 更灵活，仅为批中的每个请求执行一次迭代，即生成一个输出 token。每次迭代之后调用调度器，已完成请求离开批次，新到达请求加入，从而动态调整批次。不过，批大小仍受 GPU 内存容量限制。

3 方法

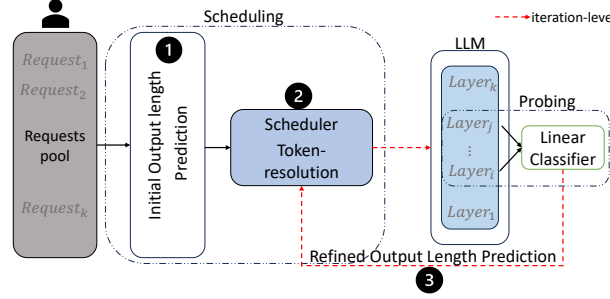


图 1: TRAIL 架构。系统 (1) 先用 BERT 模型对请求排序；(2) 采用限制抢占的 SPRPT 变体调度请求；(3) 生成 token 时，利用 LLM 内部层嵌入细化预测。每次迭代重复步骤 2 和 3，图中红色虚线表示这一循环，从而支持迭代粒度的抢占和预测更新。我们关注识别最适合预测输出长度的 LLM 层，并未使用多层嵌入 ($i = j = 11$)。图内文字译释：图内：Requests pool = 请求池；Request = 请求；Initial Output length Prediction = 初始输出长度预测；Scheduling = 调度；Scheduler Token-resolution = token 粒度调度器；Refined Output Length Prediction = 细化输出长度预测；Layer = 层；Probing = 探针分析；Linear Classifier = 线性分类器；iteration-level = 迭代级。

TRAIL 利用 LLM 推理的自回归性质和迭代级调度，通过两项主要策略降低响应时间：迭代级预测，以及考虑内存开销的抢占。它利用目标 LLM 自身的中间层嵌入，以低开销、高精度预测输出长度，同时执行 token 粒度的有限抢占调度，控制抢占造成的内存开销。图 1 展示其架构。用户将请求提交到请求池，TRAIL 先用 BERT 根据输入提示排序（步骤 1）；这一方法仅使用提示，沿用文献 (Jin et al., 2023) 提出的 BERT 方案。根据输出长度预测，调度器执行限制抢占的最短预测剩余处理时间 (SPRPT) 变体（步骤 2，见第 3.3 节）。SPRPT 原本面向单服务器顺序调度，我们将其适配为处理多个并发请求，仍优先处理预测剩余时间最短的请求；并发请求数受可用 GPU 内存限制。LLM 每生成一个输出 token，就利用其中间层嵌入细化预测。线性分类器处理这些嵌入，将结果提供给调度器，调度器据更新后的预测调整（第 3.1 节）。

3.1 细化输出长度预测

本节介绍利用 LLM 中间层嵌入估计提示长度的方法。¹

问题定义。 设 $\mathbf{u}^{(1)}, \dots, \mathbf{u}^{(N)} \in \mathbb{R}^d$ 为模型生成的 N 个输出 token 在某个中间层的输出，其中 d 是 Transformer 隐藏维度。另以 $\mathbf{u}^{(0)}$ 表示预填充阶段的输入嵌入，由指定层上所有输入 token 的嵌入取平均得到。选择长度预测桶 $B_1, \dots, B_k \subset \mathbb{N}$ ，即对桶边界 $b_1 < b_2 < \dots < b_{k+1}$ ，有 $B_i = \{b_i, b_i + 1, \dots, b_{i+1} - 1\}$ 。训练一个线性分类器，输入嵌入 $\mathbf{u}^{(t)}$ ，输出向量 $p^{(t)} \in [0, 1]^k$ ，其中 $p^{(t)}(j)$ 表示剩余 token 数，即 $N - t$ ，落在 B_j 中的概率。

本方法重点识别最能准确预测输出长度的 LLM 层。一种可能扩展是选取多个层，对其输出加权平均，或用多个层的嵌入训练线性分类器；多层方案留待未来研究。为此，需要对每个请求在全部 LLM 层上剖析大量参数。

使用 LLama3-8b-instruct 和 Alpaca 数据集 (Taori et al., 2023)，剖析过程首先在预填充与解码阶段提取各层嵌入。对 1,000 个提示中的每个 token，记录全部 32 层的嵌入，保留嵌入张量与剩余 token 数作为训练数据。预填充结束、解码开始前的嵌入张量形状为 $[1, 44, 4096]$ ，每次解码迭代的形状则为 $[1, 1, 4096]$ 。

预测器架构。 我们根据各层嵌入训练 MLP，并评估预测精度。该神经网络由两个线性层组成的全连接网络构成：第一层将输入嵌入映射到 512 维空间，后接 ReLU；第二层将预测长度分类到 $k = 10$ 个等宽桶之一，对应 0 至 512 token 的输出长度。第 i 个桶 b_i 覆盖 $\left[\frac{512i}{10}, \frac{512(i+1)}{10}\right)$ ；最后一个桶 b_{10} 包含上界，覆盖 $[460.8, 512]$ 。

¹译注：原文此处写“提示长度”，但以下定义与实现实际预测剩余输出 token 长度；保留并指出这一用词差异。

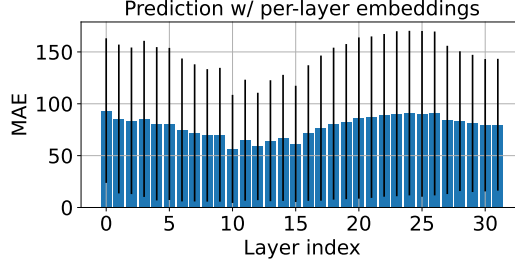


图 2: 使用不同层嵌入预测长度的平均绝对误差 (MAE)，基于 1,000 个提示。图内文字译释：图内：Prediction w/ per-layer embeddings = 使用各层嵌入的预测；Layer index = 层编号；MAE = 平均绝对误差。

训练进行 30 个 epoch，批大小为 32，使用 AdamW 优化器控制过拟合。余弦退火将学习率从 0.01 逐渐降到 0；多分类任务采用 CrossEntropyLoss 损失。这里只展示一种预测器示例，也可使用其他合适、高效的学习方法获得预测器。预测首个 token 时，对提示中所有 token 的嵌入取平均，并将平均嵌入输入预测器。全部 32 层的初步评估（图 2）表明，第 10–15 层的长度预测最准确。

重点剖析。据此，我们重点剖析这些层，收集超过 700 万对训练样本，再训练预测器，并采用平滑技术进一步提高长度预测精度。

平滑。我们观察到，单次迭代的预测概率向量虽然合理，但迭代之间可能存在较大波动。因此，采用贝叶斯推断 (Särkkä & Svensson, 2023)，在新预测到达时更新概率估计，随时间维持更准确的分布。前一次迭代的估计作为当前预测的起点；观察当前预测后，根据前一估计与当前预测的一致程度调整前一估计，再归一化。更新后的估计作为下一次迭代的起点。

每次迭代 t 都更新先验，因为剩余长度可能随时间从一个桶移到另一个桶。具体地，设 $T \in [0, 1]^{k \times k}$ 为转移矩阵， $T_{i,j}$ 表示位于迭代 $t-1$ 的桶 B_j 中的值，在迭代 t 移到桶 B_i 的概率。这里只会在相邻桶之间转移，即从 B_{i+1} 移到 B_i 。假设每个桶内长度均匀分布，故对角项 $T_{i,i}$ 表示仍留在同一桶的概率，为 $1 - \frac{1}{\text{bin size}}$ ；非对角项 $T_{i,i+1}$ 表示从 B_{i+1} 移到 B_i 的概率，为 $\frac{1}{\text{bin size}}$ 。 T 根据桶大小一次性计算，其结构见附录 A。迭代 t 的概率估计 $\hat{q}^{(t)}$ 如下计算：

1. 初始化 $\hat{q}^{(0)} = p^{(0)}$ 。
2. 每次迭代 t 更新先验： $\hat{q}_{\text{prior}}^{(t)} = T \cdot \hat{q}_{\text{prior}}^{(t-1)}$ 。
3. 更新当前概率： $\hat{q}^{(t)}(i) = \frac{\hat{q}_{\text{prior}}^{(t)}(i)p^{(t)}(i)}{\sum_j \hat{q}_{\text{prior}}^{(t)}(j)p^{(t)}(j)}$ 。

迭代 t 的预测长度为 $L_t = \sum_i \hat{q}^{(t)}(i) \cdot m_i$ ，其中 m_i 是桶 B_i 中的平均长度，即 $m_i = \frac{b_i + b_{i+1}}{2} = \frac{128(2i+1)}{5}$ 。这一迭代过程随时间细化先验分布。图 3 展示不同模型层预测的平均绝对误差。²

3.2 每次迭代计算输出长度预测

根据从 LLM 中间层（本文选择第 11 层）提取的嵌入，有两种预测计算方式。第一种直接在 GPU 上，利用指定层已经生成的嵌入计算预测；避免了数据传输，但预测与模型前向传播共享 GPU，会减慢解码。第二种在 CPU 上计算：将第 11 层嵌入传到 CPU，CPU 计算预测的同时，GPU 继续处理剩余层，即第 12–32 层。

无论在 GPU 还是 CPU 上执行长度预测，其计算开销都很小。用于长度预测的 MLP 约有 2.1 百万参数，而 Llama 模型有 8 十亿参数。每个 token 的计算量 (FLOPs) 大致与参数数目成比例，因此长度预测的开销约为 0.03%。使用更大模型或更小 MLP 时，开销占比还会下降。表 1 展示两种方式的平均长度预测推理时间。

²译注：原文更新式从上一时刻的 prior 递推，而解释称使用上一时刻更新后的估计；另外，桶编号与末桶范围之间有索引差异。此处保留公式，附录矩阵方向的差异另作说明。

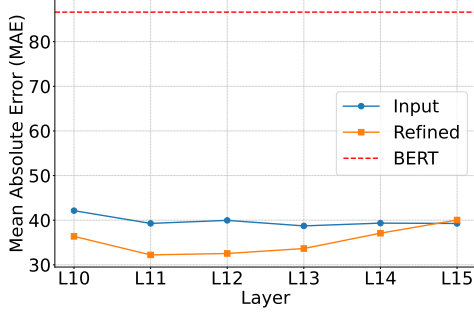


图 3: 不同层预测长度的平均绝对误差: BERT 输入嵌入预测为红色虚线, 未细化的平均 token 嵌入预测为蓝色, 细化后的预测为橙色。图内文字译释: 图内: Mean Absolute Error (MAE) = 平均绝对误差; Layer = 层; Input = 仅使用输入嵌入; Refined = 细化预测; BERT = BERT 基线; L10-L15 表示第 10-15 层。

设备	批大小	均值 (μs)	标准差 (μs)
CPU	512	9.43	3.75
CPU	1024	6.19	1.46
CPU	2048	5.94	1.09
CUDA	512	0.615	0.093
CUDA	1024	0.497	0.078
CUDA	2048	0.429	0.084

表 1: CPU 和 CUDA 在不同批大小下, 每个样本推理时间 (TPS) 的均值与标准差, 单位为微秒 (μs)。

3.3 调度策略

传统作业³调度通常假设完成时间完全未知, 因而自然使用 FCFS, 或者完成时间事先完全已知, 因而可使用最短作业优先 (SJF) 降低平均等待时间。许多实际系统却不知道精确完成时间。一些研究探索使用预测而非精确服务时间的队列系统, 通常以最小化作业在系统中的平均停留时间为目标。在已有工作基础上, 文献 (Mitzenmacher, 2019; Shahout & Mitzenmacher, 2024) 在一次只执行一个作业的单服务器环境下, 分析最短预测作业优先 (SPJF) 和最短预测剩余处理时间 (SPRPT)。SPRPT 跟踪每个作业的预测剩余处理时间, 允许服务时间预测更短的新作业到达后进行抢占。

这些工作说明了预测的价值, 却没有考虑 LLM 系统中抢占带来的内存开销, 特别是键值缓存管理问题 (第 2 节)。抢占虽然可以降低响应时间, 却增加内存消耗, 最终可能因内存约束而降低性能。LLM 系统内存耗尽时, 要么将作业从内存丢弃, 等空间可用后重新计算; 要么将 KV 缓存从 GPU 换出到 CPU。两者都会影响响应时间: 丢弃需要重新计算, 而交换会中断模型前向传播, 使整个运行批次延迟。直观上, 应限制抢占以避免超过内存约束; 尤其是执行较早阶段的抢占消耗较少资源, 而接近完成时应避免抢占。

在 TRAIL 中, 给定作业的初始预测 r , 即预测桶中点对应的数值, 仅允许它在前 $\lfloor C \cdot r \rfloor$ 次迭代中被抢占, 其中 C 为固定常数。这使作业接近完成、占用较多内存时受到抢占保护。

有趣的是, 可以分析单服务器有限抢占 SPRPT 的相应理论模型, 我们认为这一模型本身也具有研究价值。它没有内存概念, 且仅有单服务器, 不能完整刻画 LLM 系统的复杂性, 但仍为有限抢占的潜力提供见解。采用标准排队论假设, 即到达率 $\lambda < 1$ 的泊松到达、服务与预测时间独立的 M/G/1 队列, 可以推导平均响应时间的闭式表达式 (引理 1)。

具体地, 各到达作业的处理时间相互独立, 遵循累积分布 $F(x)$, 密度为 $f(x)$ 。预测在不同作业之间相互独立, 由密度函数 $g(x, r)$ 描述, 其中 x 为真实大小, r 为预测大小, 因此 $\int_{r=0}^{\infty} g(x, r) dr = f(x)$ 。作业用三元组 (x, r, a) 描述, x 为真实大小, r 为预测大小, a 为作业年龄, 即已接受服务的时间。设置阈值 $a_0 = C \cdot r$, 其中 C 可调。系统在 $a < a_0$ 时允许抢占, $a \geq a_0$ 时禁止抢占, 在早期资源占用与及时完成之间权衡。当 $C = 1$ 时, 该系统与 SPRPT 相同。

以下引理 1 的证明见附录 C。有限抢占 SPRPT 相比传统 SPRPT 改善内存使用的模拟结果见附录 D。此外, 附录 E 在相同框架下提供适用于细化预测的另一个理论模型, 它也可能具有独立研究价值。

³本文交替使用作业与请求两个术语。

引理 1. 对于有限抢占 *SPRPT*，作业在年龄 a_0 时变为不可抢占；真实大小为 x 、预测大小为 r 的作业，其期望平均响应时间为

$$\mathbb{E}[T(x, r)] = \frac{\lambda \left(\int_{y=0}^r \int_{x_I=0}^{\infty} x_I^2 \cdot g(x_I, y) dx_I dy + \int_{t=r+a_0}^{\infty} \int_{x_I=t-r}^{\infty} g(x_I, t) \cdot (x_I - (t-r))^2 \cdot dx_I dt \right)}{2(1 - \rho'_r)^2} + \int_0^{a_0} \frac{1}{1 - \rho'_{(r-a)^+}} da + (x - a_0).$$

其中 $\rho'_r = \lambda \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy$ 。

4 实验评估

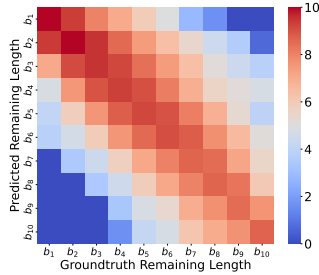
设置与实现。 实现基于开源 *vLLM* v0.5.0，本文调度器与基线均启用分块预填充。内存不足时设置为丢弃作业，待内存可用后重新计算。实验服务器配备一块 NVIDIA A100 80GB GPU、64 个 AMD EPYC 7313 16-Core Processor 核心、503 GiB 内存，运行 CUDA 12.3。单 GPU 服务模型为 *LLama3-8b-instruct*。工作负载来自 Alpaca 数据集 (Taori et al., 2023)，该数据集源于开源对话记录，最初用于微调 Alpaca 模型。模型服务采样 10,000 个不同提示，与长度预测器的训练提示不同。

基准与指标。 基准模拟客户端与服务器模式的聊天机器人服务：服务器运行 LLM，客户端按指定到达率提交请求。服务器运行 *vLLM* 的 OpenAI API，客户端提交 Alpaca 提示以模拟实际场景。基准测量延迟和首 token 时间 (TTFT) 的均值与中位数，考察不同负载下的响应能力。除常规负载外，也评估突发到达情况。

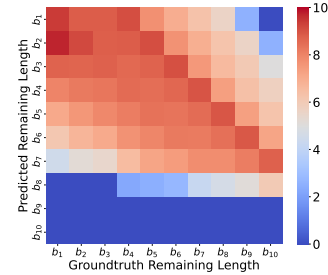
基线。 比较四种配置：(1) 原版 *vLLM* (Kwon et al., 2023)，使用先来先服务；(2) 基于 BERT 预测的最短作业优先 *vLLM*，并启用分块预填充；(3) 使用细化嵌入预测的 *TRAIL*；(4) 使用 BERT 预测的 *TRAIL*。

4.1 预测准确性

图 3 比较根据提示进行的 BERT 预测、使用不同层嵌入的线性分类器预测，以及贝叶斯细化后的预测的平均绝对误差 (MAE)。我们进一步将第 11 层的细化预测与真实值及 BERT 预测比较。热力图展示多条序列中真实剩余长度与预测剩余长度的对应关系；横轴为真实值，纵轴为预测值，均分成十个桶，表示序列推进时的剩余长度。第 i 个桶 b_i 覆盖 $\left[\frac{512i}{10}, \frac{512(i+1)}{10} \right)$ 。每个请求在执行中的每一次预测，都在热力图中计数，因此同一请求会多次出现。单元格显示不同阶段相同真实值与预测值组合的出现次数的对数。BERT 仅作一次预测，为便于比较，每生成一个 token，就从初始预测长度减去 1。图 4 表明，第 11 层嵌入的细化预测更准确：对角线上值更高，非对角线处值更低。BERT 则相反，对角线值较低、非对角线值较高，表明预测与真实长度偏差更大。



(a) 基于嵌入的预测



(b) BERT 预测

图 4: 真实长度桶与预测长度桶的对数尺度比较。第 i 个桶 b_i 覆盖 $\left[\frac{512i}{10}, \frac{512(i+1)}{10} \right)$ 。图内文字译释：图内：Groundtruth Remaining Length = 真实剩余长度；Predicted Remaining Length = 预测剩余长度；b1–b10 为长度桶；色条表示出现次数的对数。

4.2 LLM 服务

首先将请求到达率设为 14, 比较 TRAIL 在不同 c 值 ($c = 0.5, 0.8, 1$) 下的平均延迟与 TTFT。其中 $c = 1$ 模拟 unlimited 抢占的 SRPT。图 5 表明, $c = 1$ 时平均延迟和 TTFT 更高, $c = 0.8$ 时最低, $c = 0.5$ 紧随其后。我们还测试了 $c = 0.2$, 其延迟与 TTFT 均高于较大的 c ; 为更清楚展示当前 c 区间, 该结果未在图中展示。这说明抢占虽然有益, 但其额外内存开销会影响服务效率, 因此应受到限制。后续实验将 TRAIL 的默认 c 设为 0.8。不过, 最优 c 可能随系统内存与负载变化, 因为它影响抢占收益与内存开销的权衡。



图 5: 请求到达率为 14 时, 不同 c 值 ($c = 0.5, 0.8, 1$) 下的平均延迟与 TTFT 比较。图内文字译释: 图内: Mean Latency (s) = 平均延迟 (秒); Mean TTFT (ms) = 平均首 token 时间 (毫秒); 横轴为不同 c 值。

图 6 展示四种 LLM 服务系统的延迟均值与中位数, 以及 TTFT 随请求到达率的变化: (1) 原版 vLLM (Kwon et al., 2023), 采用先来先服务 (FCFS), 记为 vLLM-FCFS; (2) 对新调度序列, 按提示的 BERT 预测执行最短作业优先 (SJF) 的 vLLM, 记为 vLLM-SJF_BERT, 它还优先处理新序列并启用分块预填充; (3) $c = 0.8$ 的 TRAIL, 使用细化嵌入预测, 并启用分块预填充; (4) $c = 0.8$ 的 TRAIL, 使用 BERT 预测, 记为 TRAIL-BERT。TRAIL-BERT 与 TRAIL 的比较检验嵌入预测的效果; 两种调度策略的 vLLM 与两种预测方式的 TRAIL 之间的比较, 检验有限抢占的作用。图中, 按 BERT 预测执行 SJF, 相比 vLLM-FCFS 收益很小, 因为 vLLM 的实现优先处理新请求, 而不是已经运行的请求。采用有限抢占的 TRAIL 和 TRAIL-BERT 均获得更低的延迟均值与中位数 (图 6a、6c), 以及更低的 TTFT 均值与中位数 (图 6b、6d)。其中, 使用细化嵌入预测的 TRAIL 因预测更准确, 延迟与 TTFT 最低 (图 3、4)。

图 7 展示突发场景下, vLLM、vLLM-SJF_BERT 和两种 c 值 ($c = 0.8$ 与 $c = 1$) 的 TRAIL 的延迟均值与中位数, 以及 TTFT。实验开始时, 全部请求在极短时间内到达, 模拟需求突然上升。TRAIL 仍获得较低延迟与 TTFT, 因为我们的实现按长度预测为所有运行中和等待中的请求排序, 而 vLLM 优先处理新请求。不过, 处理过程中没有新请求到达, 抢占不再具有优势, 因此 $c = 0.8$ 与 $c = 1$ 的 TRAIL 性能接近。

5 相关工作

近期已有多项工作优化大语言模型 (LLM) 服务。ORCA (Yu et al., 2022) 引入 token 级调度, 按 token 而非整条序列处理批次。vLLM (Kwon et al., 2023) 采用 PagedAttention 降低内存开销。这些设计有效, 但使用先来先服务 (FCFS), 仍可能遭遇队头阻塞。

传统调度中, 最短作业优先 (SJF) 和最短剩余处理时间 (SRPT) 旨在降低响应时间, 多级反馈队列 (MLFQ) 则在不知道作业大小时动态调整优先级。LLM 调度的一条有前景的方向是输出长度预测。FastServe (Wu et al., 2023) 在 ORCA 基础上, 分别调度每个输出 token, 通过 MLFQ 避免队头阻塞; 但这造成频繁抢占, 提高 KV 缓存管理和 CPU 卸载的成本。若干工作提出基于预测的调度来应对这些挑战。Zhen 等 (Zheng et al., 2024) 通过额外 LLM 预测响应长度, 并按预测长度调度, 从而改善推理。不过, 额外的长度预测 LLM 引入了预测开销。

其他工作利用 DistilBERT、OPT 等机器学习模型预测输出长度, 改善资源分配并减少推理内存问题。文献 (Jin et al., 2023; Stojkovic et al., 2024; Cheng et al., 2024) 将长度预测建模为分类问题, 另一些工作 (Qiu et al., 2024b,a) 采用回归方法。本文沿此方向进一步利用目标 LLM 的嵌入预测输出长度, 并通过限制抢占改善调度效率。

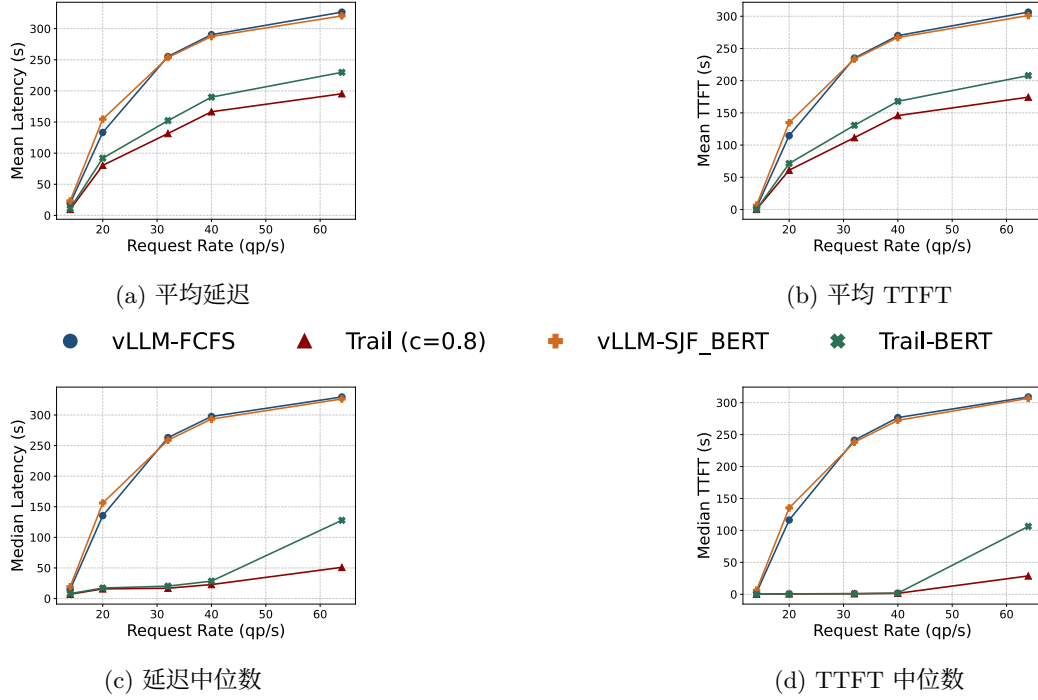


图 6: 四种 LLM 系统的延迟均值与中位数以及 TTFT 随请求到达率的变化: (1) 采用 FCFS 的 vLLM-FCFS; (2) 基于 BERT 预测执行 SJF 的 vLLM-SJF_BERT; (3) $c = 0.8$ 、使用细化嵌入预测的 TRAIL; (4) $c = 0.8$ 、使用 BERT 预测的 TRAIL-BERT。图内文字译释: 图内: Request Rate (qp/s) = 请求到达率 (每秒请求数); Mean/Median Latency (s) = 延迟均值/中位数 (秒); Mean/Median TTFT (s) = 首 token 时间均值/中位数 (秒)。图例从左到右为 vLLM-FCFS、Trail ($c=0.8$)、vLLM-SJF_BERT 与 Trail-BERT。

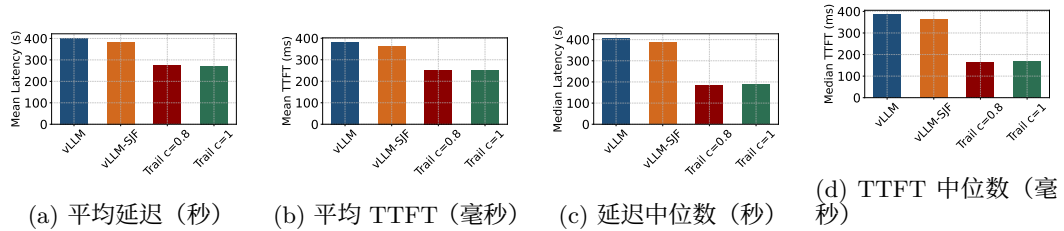


图 7: 请求突发到达时, 延迟与 TTFT 的均值和中位数。图内文字译释: 图内: Mean/Median Latency (s) = 延迟均值/中位数 (秒); Mean/Median TTFT (ms) = 首 token 时间均值/中位数 (毫秒)。横轴分别为 vLLM、vLLM-SJF、Trail $c=0.8$ 和 Trail $c=1$ 。

6 局限性与结论

本文提出 TRAIL, 一种显著改善 LLM 推理响应时间的新方法, 具有两项关键贡献。首先, 从目标 LLM 本身获取低开销、高精度预测。每生成一个输出 token, 就复用其内部结构的嵌入, 输入轻量分类器, 预测每个运行请求的剩余长度。相比仅依赖序列提示的现有预测方法, 该方法更准确、开销更低; 具体而言, 基于层嵌入的细化预测, 将平均绝对误差降至序列提示 BERT 预测的 $1/2.66$ 。其次, 利用输出长度预测提出有限抢占 SRPT 变体, 专门考虑 LLM 系统的内存开销。我们在 M/G/1 队列模型下推导其闭式公式, 并通过模拟展示潜力。将 TRAIL 集成到 vLLM 后, 实际数据集实验显示整体延迟和首 token 时间改善。在 Alpaca 数据集上, 相比 vLLM, 平均延迟降至其 $1/1.66$ 至 $1/2.01$, 平均首 token 时间降至其 $1/1.76$ 至 $1/24.07$ 。

局限性。关键挑战之一, 是选择信息最丰富的层嵌入作为线性分类器输入。剖析全部层的嵌入计算开销较大, 受时间与资源限制, 我们仅研究 Alpaca 数据集。未来有几条值得探索的方向: 在多个数据集上研究层选择与预测精度的关系; 对多层嵌入加权平均, 提高精度; 尝试对数宽度的分类桶; 仅在特定间隔计算嵌入预测, 而非每次迭代都计算, 以降低开销; 还可动态重新训练分类器, 应对数据漂移并维持长期性能。

致谢

感谢 Sham Kakade 提供 Kempner 集群用于实验, 也感谢 Yonatan Belinkov 的有益讨论。Rana Shahout 的工作部分得到 Schmidt Futures Initiative 和 Zuckerman Institute 的支持。Michael Mitzenmacher 的工作部分得到 NSF 项目 CCF-2101140、CNS-2107078 和 DMS-2023528 的支持。

参考文献

- Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022.
- Ke Cheng, Wen Hu, Zhi Wang, Peng Du, Jianguo Li, and Sheng Zhang. Enabling efficient batch serving for llama via generation length prediction. *arXiv preprint arXiv:2406.04785*, 2024.
- John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. *arXiv preprint arXiv:1909.03368*, 2019.
- John Hewitt and Christopher D Manning. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4129–4138, 2019.
- Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. s^3 : Increasing gpu utilization during generative inference for higher throughput. *Advances in Neural Information Processing Systems*, 36:18015–18027, 2023.
- Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for $\{\mu\text{second-scale}\}$ tail latency. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pp. 345–360, 2019.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- Michael Mitzenmacher. Scheduling with predictions and the price of misprediction. *arXiv preprint arXiv:1902.00732*, 2019.
- OpenAI. Introducing chatgpt. <https://openai.com/blog/chatgpt>, 2022.
- Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Power-aware deep learning model serving with $\{\mu\text{-Serve}\}$. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 75–93, 2024a.
- Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Efficient interactive llm serving with proxy model-based sequence length prediction. *arXiv preprint arXiv:2404.08509*, 2024b.
- V Sanh. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- Simo Särkkä and Lennart Svensson. *Bayesian filtering and smoothing*, volume 17. Cambridge university press, 2023.
- Ziv Scully and Mor Harchol-Balter. Soap bubbles: Robust scheduling under adversarial noise. In *2018 56th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pp. 144–154, 2018.
- Rana Shahout and Michael Mitzenmacher. Skippredict: When to invest in predictions for scheduling. *arXiv preprint arXiv:2402.03564*, 2024.
- Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency. *arXiv preprint arXiv:2408.00741*, 2024.

- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, 2022.
- Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response length perception and sequence scheduling: An llm-empowered llm inference pipeline. *Advances in Neural Information Processing Systems*, 36, 2024.

附录

附录目录

A 贝叶斯推断转移矩阵	12
B SOAP 分析	12
C 有限抢占 SPRPT	13
D 有限抢占 SPRPT 的模拟	15
E 细化预测框架	16
E.1 使用细化预测的 SPRPT	17

A 贝叶斯推断转移矩阵

转移矩阵根据每个桶内均匀分布的假设，描述相邻桶间的转移。对角项和次对角项分别表示仍处于原桶，以及移动到相邻较低桶的概率。以下给出一般桶数下的矩阵形式。⁴

$$T = \begin{bmatrix} 1 - \frac{1}{\text{桶宽}} & 0 & 0 & \dots & 0 \\ \frac{1}{\text{桶宽}} & 1 - \frac{1}{\text{桶宽}} & 0 & \dots & 0 \\ 0 & \frac{1}{\text{桶宽}} & 1 - \frac{1}{\text{桶宽}} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 - \frac{1}{\text{桶宽}} \end{bmatrix}$$

B SOAP 分析

我们使用 SOAP 框架 (Scully & Harchol-Balter, 2018) 推导平均响应时间的精确表达式。SOAP 还给出响应时间分布的拉普拉斯-斯蒂尔切斯变换，但为方便全文比较，我们主要考察平均响应时间。

SOAP 用于分析可以用排名函数描述的 M/G/1 队列调度策略。Scully 和 Harchol-Balter (Scully & Harchol-Balter, 2018) 将许多调度策略归入 SOAP 类。这类策略按排名调度作业，始终优先执行排名数值最低的作业；多个作业排名相同时，以先来先服务打破平局。排名函数是关键组成部分，根据作业的静态属性分配排名，这些属性通常称为作业类型或描述符。例如，多类别模型中，描述符可以表示作业类别，也可包含服务时间，即作业大小等静态特征。排名还可能依赖作业年龄，也就是已经获得服务的时间。

SOAP 假设作业排名仅由自身属性和年龄决定，这与 TRAIL 的模型及调度算法相符。更详细的说明见文献 (Scully & Harchol-Balter, 2018)。

SOAP 采用标记作业分析法：跟踪大小为 x_J 、描述符为 d_J 的标记作业 J ，用 a_J 表示 J 已获得服务的时间。 J 的平均响应时间等于等待时间与驻留时间之和：等待时间为到达首次服务之前的时间，驻留时间为首次服务到完成之间的时间。计算等待时间时，SOAP 同时考虑在 J 之前到达的旧作业，以及 J 之后到达的新作业造成的延迟。因为排名随时间变化，一个关键概念是作业未来的最差排名。描述符为 d_J 、年龄为 a_J 的作业，其未来最差排名记为 $\text{rank}_{d_J}^{\text{worst}}(a_J)$ 。当 $a_J = 0$ 时，记作 $r_{\text{worst}} = \text{rank}_{d_J}^{\text{worst}}(0)$ 。

在 SOAP 框架中，等待时间对应变换后的 M/G/1 队列忙期，到达率为 λ ，作业大小由 $X^{\text{new}}[\text{rank}_{d_J}^{\text{worst}}(a)]$ 描述。随机变量 $X^{\text{new}}[\text{rank}_{d_J}^{\text{worst}}(a)]$ 表示新到达作业从开始服务，直到

⁴译注：正文定义的是从后一桶转向前一桶的上对角项，而下列原矩阵将非零项写在下对角线。两者方向不一致，且端点概率质量处理未展开；此处保留原矩阵。

完成或其排名超过 $rank_{d_J}^{\text{worst}}(a)$ 的服务时间。该忙期的初始工作量对应旧作业造成的延迟。为计入这部分延迟，SOAP 按排名将作业分类并变换系统。超过阈值 r_{worst} 的旧作业归为丢弃作业，不纳入变换后的系统。排名低于或等于 r_{worst} 的旧作业称为原始作业，视为以速率 λ 到达，大小分布为 $X_0^{\text{old}}[r_{\text{worst}}]$ 。随机变量 $X_0^{\text{old}}[r_{\text{worst}}]$ 表示相对于排名 r_{worst} ，原始作业提供的服务时间。曾经高于阈值、随后又降到阈值以下的再循环作业，视为长度为 $X_i^{\text{old}}[r_{\text{worst}}]$ 的服务器休假。随机变量 $X_i^{\text{old}}[r_{\text{worst}}]$ 表示相对于 r_{worst} ，作业第 i 次再循环时的服务时间，其中 $i \geq 1$ 。在 TRAIL 中，作业仅再循环一次，因此只考虑 $X_1^{\text{old}}[r_{\text{worst}}]$ 。

SOAP 利用“泊松到达看到时间平均值”性质，证明旧作业造成的稳态延迟，与变换后的 M/G/1/FCFS 系统中的排队时间同分布。该系统含稀疏服务器休假，原始作业以速率 λ 到达，大小分布为 $X_0^{\text{old}}[r_{\text{worst}}]$ 。

定理 1 (文献 (Scully & Harchol-Balter, 2018) 的定理 5.5)。在任意 SOAP 策略下，描述符为 d 、大小为 x 的作业，其平均响应时间为：

$$\mathbb{E}[T(x, d)] = \frac{\lambda \cdot \sum_{i=0}^{\infty} \mathbb{E}[X_i^{\text{old}}[r_{\text{worst}}]^2]}{2(1 - \lambda \mathbb{E}[X_0^{\text{old}}[r_{\text{worst}}])(1 - \lambda \mathbb{E}[X^{\text{new}}[r_{\text{worst}}])]} + \int_0^x \frac{1}{1 - \lambda \mathbb{E}[X^{\text{new}}[rank_{d_J}^{\text{worst}}(a)]]} da.$$

C 有限抢占 SPRPT

首先描述作业的排名函数。作业描述符为 (x, r, a) ，其中 x 是真实大小， r 是预测大小， a 是作业年龄。停止允许抢占的阈值设为 $a_0 = C \cdot r$ 。排名函数为：

$$rank(x, r, a) = \begin{cases} r - a & \text{若 } a < a_0 \\ -\infty & \text{否则} \end{cases}$$

由于该排名函数单调，作业未来最差排名就是当前基于初始预测得到的排名：

$$rank_{d,x}^{\text{worst}}(a) = \begin{cases} r - a & \text{若 } a < a_0 \\ -\infty & \text{否则} \end{cases} \quad (1)$$

当 $a = 0$ 时，记作 $r_{\text{worst}} = rank_{d,x}^{\text{worst}}(0) = r$ 。

引理 1. 对于有限抢占 SPRPT，作业在年龄 a_0 时变为不可抢占；真实大小为 x 、预测大小为 r 的作业，其期望平均响应时间为

$$\mathbb{E}[T(x, r)] = \frac{\lambda \left(\int_{y=0}^r \int_{x_I=0}^{\infty} x_I^2 \cdot g(x_I, y) dx_I dy + \int_{t=r+a_0}^{\infty} \int_{x_I=t-r}^{\infty} g(x_I, t) \cdot (x_I - (t - r))^2 \cdot dx_I dt \right)}{2(1 - \rho'_r)^2} + \int_0^{a_0} \frac{1}{1 - \rho'_{(r-a)^+}} da + (x - a_0).$$

其中 $\rho'_r = \lambda \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy$ 。

证明. 为利用 SOAP 分析有限抢占 SPRPT，我们根据未来最差排名（式 equation 1），为描述符为 (x, r, a) 的标记作业 J 计算 $X^{\text{new}}[rank_d^{\text{worst}}(a)]$ 、 $X_0^{\text{old}}[r_{\text{worst}}]$ 与 $X_i^{\text{old}}[r_{\text{worst}}]$ 。

C.0.1 计算 $X^{\text{new}}[rank_{d,x}^{\text{worst}}(a)]$ ：

假设预测大小为 r_K 的新作业 K ，在 J 的年龄为 a 时到达。若 $a < a_0$ ，且 K 的预测大小小于 J 的预测剩余处理时间 $(r - a)$ ，则 K 始终比 J 优先。因此，

$$X_{x_K}^{\text{new}}[r - a] = x_K \mathbf{1}(r_K < r - a) \mathbf{1}(a < a_0)$$

$$\mathbb{E}[X^{new}[r-a]] = \int_0^{r-a} \int_{x_K=0}^{\infty} x_K \cdot g(x_K, y) dx_K dy$$

C.0.2 计算 $X_0^{\text{old}}[r_{\text{worst}}]$:

设 I 为系统中的旧作业。作业 I 是原始作业还是再循环作业，取决于其预测大小与 J 的预测大小的关系。若 $r_I \leq r$ ，则 I 直到完成都是原始作业，因为其排名从不超过 r 。

$$X_{0,x_I}^{\text{old}}[r] = x_I \mathbf{1}(r_I \leq r).$$

$$\mathbb{E}[X_0^{\text{old}}[r]] = \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy.$$

$$\mathbb{E}[(X_0^{\text{old}}[r])^2] = \int_{y=0}^r \int_{x_I=0}^{\infty} x_I^2 \cdot g(x_I, y) dx_I dy.$$

C.0.3 计算 $X_i^{\text{old}}[r_{\text{worst}}]$:

若 $r_I > r$ ，则 I 最初被丢弃，在 $r_I - a = r$ 时再循环，即年龄 $a = r_I - r$ 时；仅当 $a < a_0$ 才持续服务至完成，对应服务量为 $x_I - a_I = x_I - (r_I - r)$ ：

因此有

$$X_{1,x_I}^{\text{old}}[r] = x_I - (r_I - r).$$

对于 $i \geq 2$,

$$X_{i,x_I}^{\text{old}}[r] = 0.$$

$$\mathbb{E}[X_1^{\text{old}}[r]^2] = \int_{r_I=r+a_0}^{\infty} \int_{x_I=r_I-r}^{\infty} g(x_I, t) \cdot (x_I - (r_I - r))^2 \cdot dx_I dr_I.$$

应用 SOAP 定理 5.5 (Scully & Harchol-Balter, 2018)，得到描述符为 (r) 、大小为 x 的作业的平均响应时间。令

$$\rho'_r = \lambda \int_{y=0}^r \int_{x_I=0}^{\infty} x_I \cdot g(x_I, y) dx_I dy.$$

则有

$$\begin{aligned} \mathbb{E}[T(x, r)] &= \frac{\lambda \left(\int_{y=0}^r \int_{x_I=0}^{\infty} x_I^2 \cdot g(x_I, y) dx_I dy + \int_{t=r+a_0}^{\infty} \int_{x_I=t-r}^{\infty} g(x_I, t) \cdot (x_I - (t - r))^2 \cdot dx_I dt \right)}{2(1 - \rho'_r)^2} \\ &\quad + \int_0^{a_0} \frac{1}{1 - \rho'_{(r-a)^+}} da + (x - a_0). \end{aligned}$$

令 $f_p(y) = \int_{x=0}^{\infty} g(x, y) dx$ 。大小为 x 的作业的平均响应时间，以及所有作业的平均响应时间分别为

$$\mathbb{E}[T(x)] = \int_{y=0}^{\infty} f_p(y) \mathbb{E}[T(x, y)] dy,$$

$$\mathbb{E}[T] = \int_{x=0}^{\infty} \int_{y=0}^{\infty} g(x, y) \mathbb{E}[T(x, y)] dy dx.$$

□

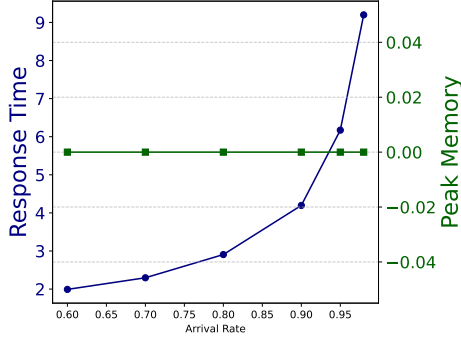
D 有限抢占 SPRPT 的模拟

模拟中，将内存占用建模为与作业年龄成比例，即与已经获得的服务量成比例。这反映了作业执行越久，占用内存越多的直觉。我们同时测量响应时间和内存消耗，以评估调度效率。

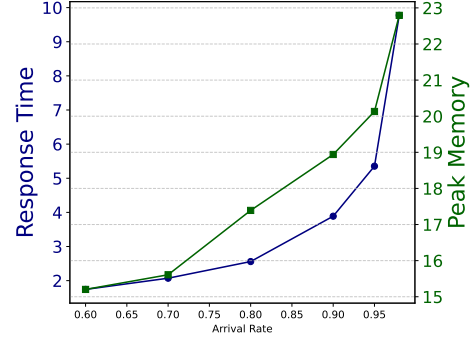
考虑泊松到达的单队列，作业服务时间服从均值为 1 的指数分布 ($f(x) = e^{-x}$)。使用两种预测模型：(1) 指数预测 (Mitzenmacher, 2019)，真实服务时间为 x 的作业，其预测也服从均值为 x 的指数分布，原文写作 $g(x, y) = e^{-\frac{x-y}{x}}$ ；(2) “完美预测器”，预测始终准确，原文写作 $g(x, y) = e^{-x}$ 。⁵

图 8 展示不同到达率与 C 值下的平均响应时间（蓝线）和峰值内存占用（绿线）。主要结论是：限制抢占，减少中断与重新调度，可以在保持合理响应时间的同时提高内存利用效率。

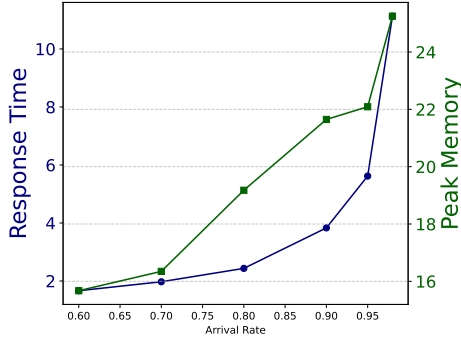
⁵译注：上述原文写法不是归一化的联合密度。第一种需区分真实大小的密度与以真实大小为条件的预测密度；第二种完美预测还应表达预测值与真实值相等的约束。此处保留公式，复现时需核查。



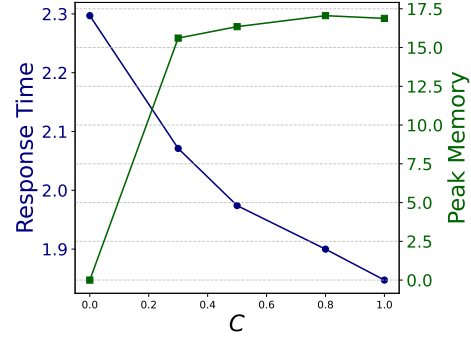
(a) $C = 0$ 时随到达率的变化



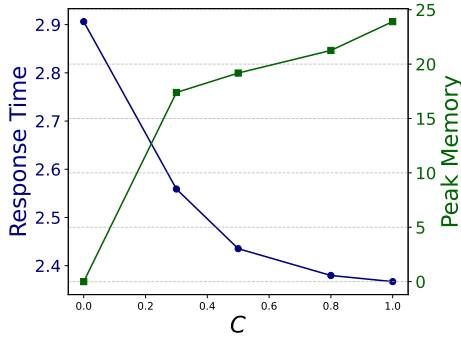
(b) $C = 0.3$ 时随到达率的变化



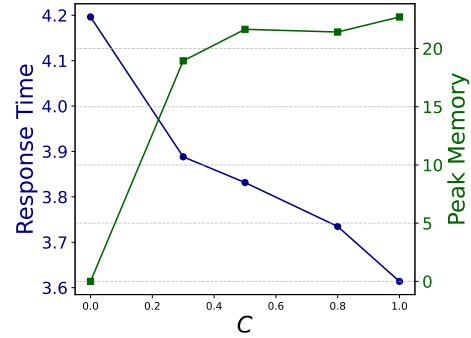
(c) $C = 0.5$ 时随到达率的变化



(d) $\lambda = 0.7$ 时随 C 的变化



(e) $\lambda = 0.8$ 时随 C 的变化



(f) $\lambda = 0.9$ 时随 C 的变化

图 8: 不同到达率与 C 值下的内存占用和响应时间比较。图内文字译释: Arrival Rate = 到达率; Response Time = 响应时间, 对应左轴蓝线; Peak Memory = 峰值内存, 对应右轴绿线; C 是抢占阈值比例。峰值内存采用本文模拟度量, 未标物理容量单位。

E 细化预测框架

考虑到达率为 λ 的 M/G/1 队列。每个到达作业的处理时间相互独立, 遵循累积分布 $F(x)$, 相应密度为 $f(x)$ 。

给定大小为 $X = x$ 的作业及其细化预测序列 $Y_0, Y_1, \dots, Y_x$, 其中 Y_i 表示处理了 x 中的 i 个单位之后的预测, 我们按以下方式建模细化预测:

给定真实大小 x 和全部历史预测, 第 i 步细化预测为 y_i 的概率记为:

$$P(Y_i = y_i \mid X = x, Y_{i-1} = y_{i-1}, Y_{i-2} = y_{i-2}, \dots, Y_0 = y_0)$$

在马尔可夫假设下, 第 i 步细化预测仅依赖真实大小 x 与前一预测 y_{i-1} :

$$P(Y_i = y_i \mid X = x, Y_{i-1} = y_{i-1}, Y_{i-2} = y_{i-2}, \dots, Y_0 = y_0) = P(Y_i = y_i \mid X = x, Y_{i-1} = y_{i-1})$$

预测序列 $Y_0, Y_1, \dots, Y_x$ 的联合概率可建模为:

$$P(Y_0 = y_0, Y_1 = y_1, \dots, Y_x = y_x \mid X = x) = P(Y_0 = y_0 \mid X = x) \prod_{i=1}^x P(Y_i = y_i \mid X = x, Y_{i-1} = y_{i-1})$$

对于预测列表 $y = [y_0, y_1, \dots, y_{x-1}]$, 其中每个 y_i 在作业 x 完成一个服务单位后得到, 原文通过对所有可能的细化预测求和, 定义相应密度 $g(x, y)$:⁶

$$g(x, y) = \sum_{y_0=0}^{\infty} \sum_{y_1=0}^{\infty} \cdots \sum_{y_{x-1}=0}^{\infty} P(Y_0 = y_0, Y_1 = y_1, \dots, Y_{x-1} = y_{x-1} \mid X = x)$$

E.1 使用细化预测的 SPRPT

相关属性为大小与细化预测列表 r 。系统可以用描述符 $\mathcal{D} = (\text{大小}, \text{细化预测列表}, \text{年龄})$ 建模。

设 J 为描述符为 (x, r, a) 的作业, x 是 J 的大小, r 是 J 的细化预测列表, 每处理一个服务单位生成一次预测, a 是 J 的年龄。使用细化预测的 SPRPT, 其排名函数为 $\text{rank}(x, r, a) = r[a] - a$, 其中作业大小为 x , 时刻 a 的预测大小为 $r[a]$ 。最大排名记为 $r_{\max}$ 。⁷

J 的未来最差排名为:

$$\text{rank}_{d,x}^{\text{worst}}(a) = \sup_{a < b < x} (r[b] - b) = \max_{a < b < x} (r[b] - b)$$

当 $a = 0$ 时, 记作 $r_{\text{worst}} = \text{rank}_{d,x}^{\text{worst}}(0) = r_{\max}$ 。

用 SOAP 分析带细化预测的 SPRPT, 需要求解多维求和。获得闭式表达式可能很困难, 因此本节提出在不依赖闭式解的情况下, 计算 SOAP 分析相关组成量的方法, 即 $X_{\text{new}}^{\text{new}}[\text{rank}_d^{\text{worst}}(a)]$ 、 $X_0^{\text{old}}[r_{\text{worst}}]$ 和 $X_i^{\text{old}}[r_{\text{worst}}]$ 。

设 J 为描述符为 (x, r, a) 的标记作业, 依据其未来最差排名进行分析。

计算 $X_{x_K}^{\text{new}}[\text{rank}_{d,x}^{\text{worst}}(a)]$: 假设预测大小为 r_K 的新作业 K , 在 J 的年龄为 a 时到达。 K 的排名低于作业 J 的最差可能排名时, 对应服务量为:

$$X_{x_K}^{\text{new}}[\text{rank}_{d,x}^{\text{worst}}(a)] = \sum_{a_K=0}^{x_K} \mathbf{1} \left(r_K[a_K] - a_K < \max_{a < b < x} (r[b] - b) \right) \cdot \mathbf{1} \left(\forall j < a_I, r_I[j] - j < \max_{a < b < x} (r[b] - b) \right)$$

其中, $\mathbf{1}(\cdot)$ 是指示函数, 条件成立时为 1, 否则为 0。

$$\begin{aligned} \mathbb{E} [X_{\text{new}}^{\text{new}} [\text{rank}_{d,x}^{\text{worst}}(a)]] &= \sum_{x_K=0}^{\infty} \left(\sum_{a_K=0}^{x_K} \mathbf{1} \left(r_K[a_K] - a_K < \max_{a < b < x} (r[b] - b) \right) \right. \\ &\quad \cdot \mathbf{1} \left(\forall j < a_I, r_I[j] - j < \max_{a < b < x} (r[b] - b) \right) \\ &\quad \cdot g(x_K, y) \Big) \end{aligned}$$

⁶译注: 下式对全部预测路径的条件概率求和, 若分布归一化, 结果为 1; 它没有保留指定路径, 也未包含作业大小的密度, 因此并不能直接作为所称联合密度。译文保留原式, 不将其作为已经验证的概率模型。

⁷译注: 正文预测的是剩余长度, 而这里的排名又减去年龄; 若预测列表记录剩余长度, 会重复扣除已服务量。要与公式一致, 需将列表解释为总大小预测, 或另行核对定义。

其中密度函数 $g(x_K, y)$ 定义为:

$$g(x_K, y) = \sum_{y_0=0}^{\infty} \sum_{y_1=0}^{\infty} \cdots \sum_{y_{x_K-1}=0}^{\infty} P(Y_0 = y_0, Y_1 = y_1, \dots, Y_{x_K-1} = y_{x_K-1} \mid X = x_K)$$

计算 $X_0^{\text{old}}[r_{\text{worst}}]$: 设 I 为系统中的旧作业。作业 I 是否是原始作业, 取决于其预测大小与 J 的预测大小的关系。 I 在排名超过 r_{max} 前为原始作业; 一旦遇到某个年龄, 使 $r_I[a_I] - a_I > r_{\text{max}}$, 就停止求和:

$$X_{0,x_I}^{\text{old}}[r_{\text{max}}] = \sum_{a_I=0}^{x_I} \mathbf{1}(r_I[a_I] - a_I < r_{\text{max}}) \cdot \mathbf{1}(\forall j < a_I, r_I[j] - j < r_{\text{max}})$$

计算 $X_i^{\text{old}}[r_{\text{worst}}]$: 若 $r_I > r$, 则 I 最初被丢弃, 在 $r_I[a] - a = r_{\text{worst}}$ 时再循环。也就是说, 年龄达到 $a = r_I[a_I] - r_{\text{worst}}$ 时开始再次服务, 直到排名又高于 r_{worst}

设 i 为区间索引。作业 I 的第 i 个区间, 定义为作业 I 排名低于 r_{max} 的年龄区间。具体如下:

$$b_0[r] = 0$$

这表示第一个区间从年龄 0 开始。

$$c_0[r] = \inf\{a > 0 \mid r_I(a) - a > r_{\text{max}}\}$$

其中, $c_0[r]$ 为大于 0 的最小年龄, 在该年龄排名函数 $r_I(a) - a$ 超过 r_{max} , 从而结束第一个排名低于 r_{max} 的区间。

对于后续区间 $i \geq 1$:

$$b_i[r] = \inf\{a \geq c_{i-1}[r] \mid r_I(a) - a < r_{\text{max}}\}$$

这是第 i 个区间的起点; 在前一区间 $[b_{i-1}[r], c_{i-1}[r]]$ 结束后, 一旦排名 $r_I(a) - a$ 再次降至 r_{max} 以下, 新区间就开始。

$$c_i[r] = \inf\{a > b_i[r] \mid r_I(a) - a > r_{\text{max}}\}$$

其中, $c_i[r]$ 为第 i 个区间的终点, 即排名 $r_I(a) - a$ 再次超过 r_{max} 的位置。

对于 $i \geq 0$, 第 i 个区间的工作量是随机变量 $X_i^{\text{OLD}}[r]$, 表示作业 I 在其第 i 个区间中, 排名低于 r_{max} 时的 token 数总和。

$$X_i^{\text{OLD}}[r] = \begin{cases} 0 & \text{若 } X_d < b_i[r] \\ X_d - b_i[r] & \text{若 } b_i[r] \leq X_d < c_i[r] \\ c_i[r] - b_i[r] & \text{若 } c_i[r] \leq X_d \end{cases}$$

若 $b_i[r] = c_i[r] = \infty$, 定义 $X_i^{\text{OLD}}[r] = 0$ 。